FACULTAD DE INFORMÁTICA
MÁSTER DE INGENIERÍA DEL SOFTWARE



SolidarianID

Memoria de Arquitectura de Datos

Grupo 3:
Antonio Pérez Serrano
Fabián Sola Durán
Manuel Bernabé Rodríguez
Pablo Peris Solano
Sergio Ros Liarte

Arquitectura de Datos
12/01/2026

Índice

1. Introducción **4**

2. Diseño conceptual de datos **4**
2.1. Diagrama de Entidad-Relación 4
2.2. Explicación de entidades, relaciones y atributos clave 5
2.2.1. Entidad *USUARIO* y especializaciones 6
2.2.2. Entidad *ADMIN_DE_LA_PLATAFORMA* 6
2.2.3. Entidad *COMUNIDAD* 6
2.2.4. Entidad *CAUSA* 7
2.2.5. Entidad *ACCION* 7
2.2.6. Entidad *EVENTO* 7
2.2.7. Entidad *LUGAR* 7
2.2.8. Entidad *OBJETIVO* 7
2.2.9. Entidad *SOLICITUD* y especializaciones 8

3. Diseño e implementación de la arquitectura de datos **8**
3.1. Elección de sistemas de almacenamiento 8
3.2. Comparativa entre las opciones consideradas 9
3.2.1. *Core* de la aplicación: *PostgreSQL vs MySQL* 9
3.2.2. Base documental para admin: MongoDB vs Cassandra 10
3.2.3. Plataforma de *streaming* de eventos: *RabbitMQ vs Kafka* 10

4. Diseño de la arquitectura **11**
4.1. Diagrama de la arquitectura final en el MVP 11
4.2. Explicación de los componentes y su integración 12

5. Implementación de la arquitectura **12**
5.1. Descripción de la implementación de cada sistema de base de datos . . . 13
5.2. Mecanismos implementados o que se podrían implementar 13
5.2.1. Consistencia de datos 13
5.2.2. Replicación y particionamiento 14
5.2.3. Escalabilidad y tolerancia a fallos 15

6. Consultas y procesamiento de datos **16**
6.1. Ejemplos de consultas para los casos de uso principales de MongoDB . . 16
6.1.1. Listado de comunidades pendientes (findPendingCommunityRequests) 16
6.1.2. Agrupación de Causas por ODS (findCausesGroupByOds) 17
6.1.3. Agrupación de Usuarios por País (findUsersGroupByCountry) . . 17
6.1.4. Listado de causas pendientes (findPaginated / findPendingCause-Requests) 18
6.2. Ejemplos de consultas para los casos de uso principales de PostgreSQL . 19
6.2.1. Ranking de comunidades por usuarios (findOrderByMostUsers) . 19
6.2.2. Causas por ID de comunidad (findAllByCommunityId) 19
6.2.3. Verificar existencia por móvil (existsByMobilePhone) 19
6.2.4. Estadística de nuevos miembros (findJoinedUsersLastMonths) . . 20
6.3. Explicación de cómo cada consulta optimiza el rendimiento 20

7. Descripción de configuración del Docker e instrucciones de despliegue 21

8. Conclusiones 22

A. Arquitectura completa de SolidarianID del grupo 3 24
A.1. Principios de arquitectura de datos 24
A.2. Capa transaccional del *core*: Users y Communities 25
A.3. Aplicación de administración y proyecciones analíticas 25
A.4. Dominios extendidos: redes sociales y mensajería 25
A.5. API Gateway y gobierno del acceso a datos 26
A.6. Consistencia e integración global 26

Índice de figuras

1. Diagrama ER para el caso de estudio de SolidarianID 5
2. Diagrama de la arquitectura del MVP de SolidarianID 11
3. Arquitectura final y completa planteada para SolidarianID: microservicios, bases de datos, cachés y bus de eventos. 24

1. Introducción

En este trabajo se describe el proceso de diseño e implementación de la arquitectura de datos de SolidarianID, una plataforma basada en microservicios cuyo objetivo es gestionar usuarios, comunidades y acciones solidarias, así como proporcionar una aplicación de administración con métricas e informes. El propósito principal es justificar las decisiones de almacenamiento adoptadas y demostrar cómo estas permiten cubrir los requisitos funcionales y no funcionales del sistema, especialmente en términos de consistencia, rendimiento y evolución del modelo de datos.

Para ello, en primer lugar se presenta el diseño conceptual mediante un modelo entidad–relación, identificando las entidades principales, sus relaciones y los atributos más relevantes. A continuación, se aborda el diseño e implementación de la arquitectura de datos, donde se justifica la elección de distintos sistemas de almacenamiento y se comparan alternativas para cada necesidad. Con base en estas decisiones, se define la arquitectura final y se explica la integración entre componentes, incluyendo el flujo de datos entre el *core* transaccional, las proyecciones de lectura y las capas de cache.

Posteriormente, se detalla la implementación práctica de cada tecnología seleccionada y se discuten mecanismos que pueden aplicarse para mejorar la consistencia de datos, la escalabilidad y la tolerancia a fallos. Finalmente, se incluyen ejemplos de consultas representativas de los casos de uso principales, junto con una breve explicación de cómo el diseño favorece el rendimiento. El documento se completa con las instrucciones de despliegue mediante Docker (y el enlace al repositorio GitHub con los ficheros necesarios), y concluye con un apartado final de conclusiones.

En la bibliografía se detalla el enlace [1] a nuestro repositorio de GitHub en el que se puede encontrar la carpeta `ado` con los distintos detalles relativos a la asignatura.

2. Diseño conceptual de datos

En este apartado se presenta el diseño conceptual de datos del MVP de *SolidarianID*, centrado principalmente en las funcionalidades de administración (moderación y gestión del contenido) que soporta la plataforma. El objetivo de este diseño es capturar, a nivel conceptual y de forma independiente de la tecnología, las entidades del dominio implementadas en los distintos microservicios(comunidades, causas, acciones, eventos, objetivos y solicitudes), así como las relaciones necesarias para garantizar consistencia e integridad en los procesos clave del administrador: alta y mantenimiento del contenido, control de pertenencias/roles y validación o denegación de solicitudes.

2.1. Diagrama de Entidad-Relación

El diagrama Entidad–Relación presentado en la Figura 1 sintetiza la estructura de información que manejan los microservicios en conjunto y cómo se conectan sus conceptos

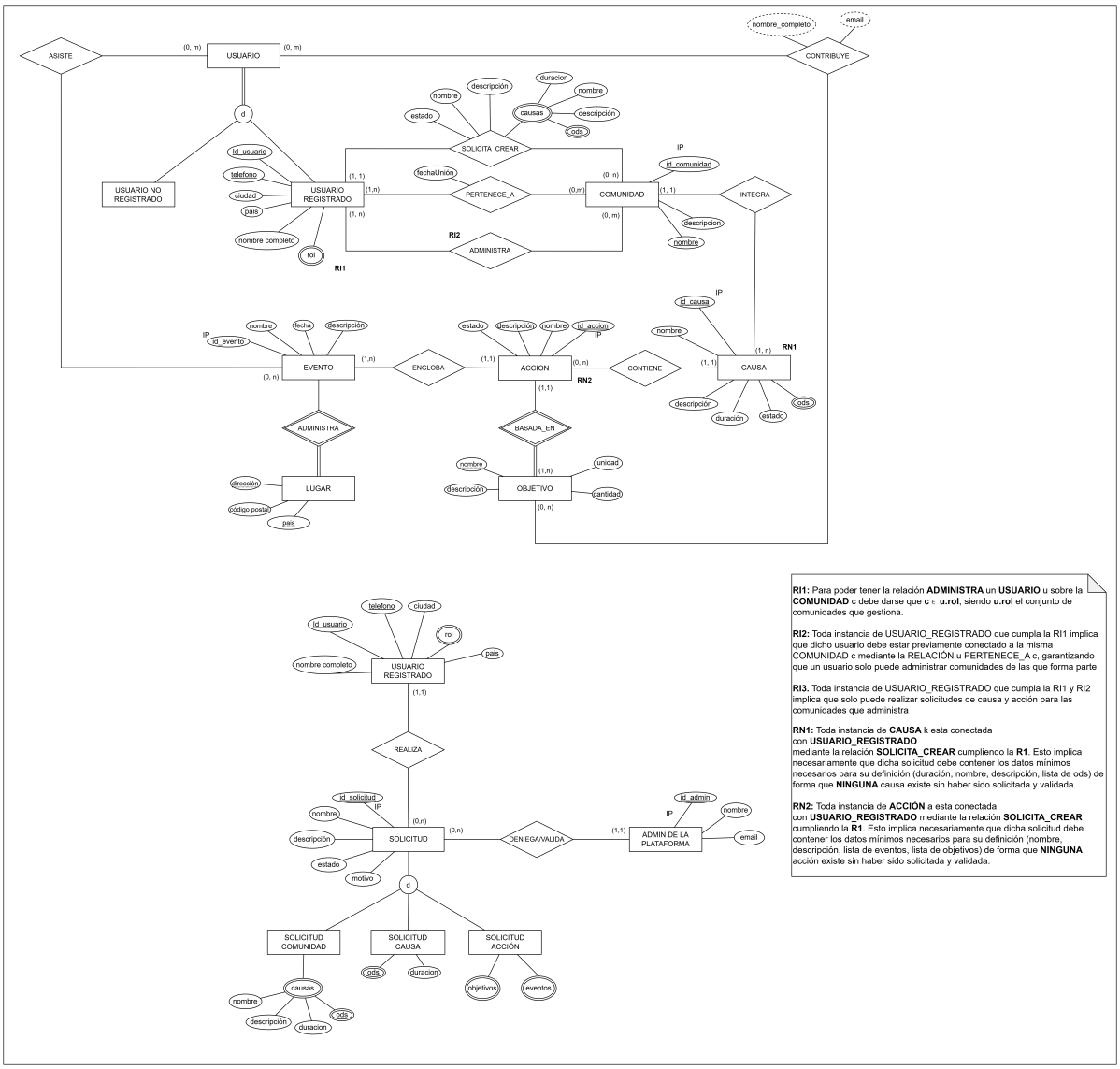


Figura 1: Diagrama ER para el caso de estudio de SolidarianID

principales. En el contexto del MVP, el diagrama pone el foco en el flujo administrativo: la creación y validación de *solicitudes* para incorporar nuevas comunidades, causas o acciones; la organización jerárquica (comunidad–causa–acción) y su trazabilidad mediante eventos, lugares y objetivos; y las restricciones de integridad tanto funcionales como no funcionales (RI/RN) que aseguran que únicamente usuarios con el rol adecuado puedan administrar comunidades y promover contenido dentro de los límites definidos.

2.2. Explicación de entidades, relaciones y atributos clave

En el modelo conceptual del MVP de *SolidarianID* las entidades se han definido para soportar, principalmente, el ciclo de vida administrativo del contenido: creación controlada (mediante solicitudes), validación/denegación y posterior organización jerárquica en *comunidades*, *causas* y *acciones*. A continuación se describe de forma íntegra cada entidad identificada en el diagrama ER, detallando su propósito dentro del dominio, sus atributos

más relevantes y el papel que desempeña en los casos de uso del administrador.

2.2.1. Entidad *USUARIO* y especializaciones

La entidad *USUARIO* representa a cualquier persona que interactúa con la plataforma, ya sea de forma anónima o autenticada. Se modela como supertipo para centralizar la información común y permitir especialización según su nivel de acceso. Sus atributos clave incluyen *id_usuario* como identificador, *nombre_completo* para la identificación funcional del usuario, y datos de contacto/localización como *teléfono*, *ciudad* y *país*. Además, incorpora el atributo *rol*, que es fundamental en el MVP orientado al administrador, ya que condiciona los permisos de gestión dentro de comunidades (por ejemplo, administrador/miembro) y sirve de base para las restricciones de integridad que limitan quién puede administrar y solicitar altas de contenido.

La especialización *USUARIO_NO_REGISTRADO* identifica al usuario que navega o consume información sin haber completado un proceso de registro/autenticación. Este subtipo permite representar explícitamente un perfil con capacidades limitadas (consumo de información y, según el diseño, participación mínima), separándolo así del usuario autenticado sin duplicar atributos comunes.

Por su parte, *USUARIO_REGISTRADO* representa al usuario autenticado que puede operar sobre funcionalidades avanzadas del sistema. Este subtipo es el que participa en los flujos clave del MVP: pertenencia a comunidades, administración de comunidades cuando su rol lo permite y realización de solicitudes para crear contenido (comunidades/causas/acciones).

2.2.2. Entidad *ADMIN_DE_LA_PLATAFORMA*

La entidad *ADMIN_DE_LA_PLATAFORMA* modela al actor con capacidad de supervisión global sobre el contenido del sistema, siendo el núcleo del enfoque del MVP. Se identifica mediante *id_admin* y se describe con atributos de identificación/contacto como *nombre* y *email*. Su función principal es gobernar el flujo de alta de contenido mediante la revisión y resolución de solicitudes realizadas por parte de usuarios registrados.

2.2.3. Entidad *COMUNIDAD*

La entidad *COMUNIDAD* representa el contenedor organizativo de primer nivel dentro del dominio. Se identifica por *id_comunidad* y dispone de atributos de descripción como *nombre* y *descripción*. En el MVP, la comunidad es el punto de partida para la gestión administrativa: agrupa a usuarios con distintos roles, organiza el contenido solidario y actúa como ámbito donde se estructuran *causas* y, por extensión, *acciones*. Esta entidad permite reflejar una agregación del contenido, facilitando la moderación por parte de usuarios administradores de comunidad.

2.2.4. Entidad *CAUSA*

La entidad *CAUSA* modela una iniciativa o problema solidario concreto dentro de una comunidad. Se identifica mediante *id_causa* y se caracteriza por atributos como *nombre* y *descripción*. Para capturar su planificación y seguimiento, incorpora atributos como *duración* y *estado*, así como *ods* (Objetivos de Desarrollo Sostenible) para clasificar su impacto y alineación con metas globales. En el contexto del MVP, la *CAUSA* es una pieza central del contenido moderable.

2.2.5. Entidad *ACCION*

La entidad *ACCION* representa la materialización operativa de una causa mediante actividades concretas, medibles y planificables. Se identifica por *id_accion* y se describe con *nombre* y *descripción*, incorporando también *estado* para reflejar su situación (por ejemplo, planificada, activa, finalizada). En el MVP enfocado al administrador, las acciones constituyen el elemento que aglutina la ejecución: suelen descomponerse en eventos (fechas concretas) y se orientan a objetivos cuantificables.

2.2.6. Entidad *EVENTO*

La entidad *EVENTO* modela una ocurrencia temporal asociada a una acción (por ejemplo, una jornada de recogida, un taller o una campaña puntual). Se identifica mediante *id_evento* y se define con *nombre*, *fecha* y *descripción*. En el MVP, el evento sirve para calendarizar la actividad solidaria y habilitar la participación de usuarios (asistencia).

2.2.7. Entidad *LUGAR*

La entidad *LUGAR* representa la localización física donde se desarrolla un evento. Se caracteriza por atributos como *dirección*, *codigo_postal* y *país*, permitiendo ubicar de forma precisa la actividad. En un MVP centrado en administración, disponer de esta entidad (o su equivalente embebido) es importante para asegurar que los eventos publicados cuentan con información mínima verificable y útil para los participantes, reduciendo ambigüedades y mejorando la calidad de los datos gestionados por la plataforma.

2.2.8. Entidad *OBJETIVO*

La entidad *OBJETIVO* modela un resultado esperado y cuantificable dentro del marco de una acción (por ejemplo, "recolectar 500 kg", "conseguir 300 firmas" o "recaudar 2 000 €"). Se describe con atributos como *nombre* y *descripción*, y se cuantifica mediante *cantidad* y *unidad*.

2.2.9. Entidad *SOLICITUD* y especializaciones

La entidad *SOLICITUD* formaliza el mecanismo de control de creación de contenido, especialmente relevante en el MVP orientado a administración. Se identifica por *id_ solicitud* y contiene atributos generales como *nombre*, *descripción*, *estado* (por ejemplo, pendiente, validada, denegada) y *motivo* para registrar la razón de una denegación o decisión administrativa. Asimismo, incorpora un discriminador (d) para especializar la solicitud según el tipo de contenido solicitado, manteniendo una base común trazable y auditable.

La especialización *SOLICITUD_ COMUNIDAD* recoge la información mínima para proponer una nueva comunidad, incluyendo atributos de definición (por ejemplo, *nombre*, *descripción*) y metadatos asociados como *causas* y *ods* cuando el diseño contempla que la comunidad se presente junto con una propuesta inicial de estructura/temáticas. La especialización *SOLICITUD_ CAUSA* representa la petición para crear una causa dentro del ámbito de una comunidad, incorporando datos típicos de planificación y clasificación como *duración* y *ods*. Por último, *SOLICITUD_ ACCION* modela la creación de una acción asociada a una causa, permitiendo adjuntar elementos operativos como *objetivos* y *eventos* propuestos. En conjunto, estas especializaciones sostienen el flujo administrativo del MVP: ningún contenido estructural entra en el sistema sin haber pasado por el circuito de solicitud y resolución, garantizando control del dominio.

3. Diseño e implementación de la arquitectura de datos

A continuación, plantearemos las justificaciones de nuestras elecciones para el almacenamiento y realizaremos una posterior comparativa de las mismas.

3.1. Elección de sistemas de almacenamiento

En este apartado se justificará el uso de cada una de las bases de datos utilizadas:

- ***PostgreSQL***: Se ha decidido utilizar esta base de datos relacional para tener consistencia en el núcleo transaccional del *MVP* (integridad referencial), es decir, todo lo relacionado con usuarios, roles, comunidades, causas, acciones solidarias, objetivos solidarios, etc. Otra consideración relevante es que las entidades identificadas en el enunciado de la práctica están claramente definidas, por lo que no presentan ambigüedades conceptuales, lo que permite un diseño de tablas coherente y preciso, garantizando una correcta representación de los datos y de las relaciones entre ellos. También aprovecharemos el uso de transacciones *ACID* para garantizar la consistencia en situaciones críticas como por ejemplo las donaciones.

- ***MongoDB***: Se ha decidido utilizar esta base de datos documental para la aplicación de administración debido a la flexibilidad que ofrece para representar las vistas administrativas. Estas constituyen proyecciones reducidas y agregadas de entidades de la base de datos principal (*PostgreSQL*), como usuarios y su uso de la platafor-

ma. Además, la flexibilidad que aporta *MongoDB* nos permite cambiar los datos a mostrar en el *dashboard* del administrador, ya que estos pueden ser cambiantes en el tiempo, al no estar bien definidos en el enunciado de la práctica. Este enfoque permite optimizar las consultas, adaptarnos a datos cambiantes en el tiempo y facilitar la evolución de dichas vistas. Un ejemplo seria el modelado de solicitudes o un diseño reducido de los usuarios en la vista de administrador.

- ***Kafka***: Se ha decidido utilizar esta base de datos como plataforma de mensajería y transmisión de eventos para la comunicación entre microservicios. Esta elección se debe a la necesidad de desacoplar los distintos componentes del sistema y permitir la propagación de eventos de dominio de forma fiable, facilitando la sincronización entre la base de datos transaccional y las distintas proyecciones de datos (por ejemplo la *MongoDB* del administrador) con las solicitudes o generación de pdfs con información de los usuarios. Este enfoque permite además escalar el sistema y reconstruir vistas de lectura de manera eficiente.

3.2. Comparativa entre las opciones consideradas

3.2.1. *Core* de la aplicación: *PostgreSQL vs MySQL*

PostgreSQL (seleccionada). PostgreSQL destaca por su robustez transaccional, su riqueza de índices y sus capacidades avanzadas de base de datos, incluyendo un amplio soporte para tipos de datos complejos, control de concurrencia multiversión (MVCC) y vistas materializadas. Estas características hacen que PostgreSQL sea especialmente adecuado para aplicaciones empresariales con operaciones de escritura frecuentes y consultas complejas, como las que se requieren en SolidarianID (gestión de relaciones, validación de entidades, roles y trazabilidad) y reducen el riesgo de inconsistencia de datos en transacciones críticas.

MySQL. MySQL es una base de datos relacional ampliamente usada, con una buena reputación en entornos de lectura intensiva y aplicaciones web sencillas. Aunque es válida para operaciones CRUD transaccionales, su soporte limitado para características avanzadas de base de datos como índices complejos, MVCC completo y tipos de datos especializados lo sitúa por detrás de PostgreSQL en escenarios que requieren mayor capacidad de consulta, modelado y escalabilidad de datos.

Conclusión. Se elige PostgreSQL como base del *core* por su equilibrio entre consistencia transaccional, expresividad de consultas. Las vistas materializadas y la adición de tipos complejos como los enumerados facilitados por la herramienta nos permiten mejorar el desarrollo de nuestra herramienta [2].

3.2.2. Base documental para admin: MongoDB vs Cassandra

MongoDB (seleccionada). Se prioriza por su modelo documental flexible y su capacidad para construir **proyecciones denormalizadas** y agregaciones para la vista de administración (estadísticas, informes, progreso) y para el historial por ID único. El caso demanda precisamente historiales completos y paneles de estadísticas e informes, que se benefician de pipelines sobre datos listos para consulta.

Cassandra (alternativa). Sobresale en cargas de escritura muy altas y escalado distribuido masivo con consistencia eventual, pero exige un modelado muy orientado a patrones de consulta predefinidos y aumenta la complejidad operativa.

Conclusión. Aunque Cassandra destaca por su elevada disponibilidad y su arquitectura distribuida basada en múltiples nodos maestros, lo que la convierte en una opción idónea para sistemas con requisitos extremos de tiempo de actividad y grandes volúmenes de escritura, estas características no son prioritarias en el contexto de la vista de administración del MVP. En SolidarianID, dicha vista trabaja principalmente con datos dinámicos, proyecciones agregadas y modelos de lectura cuya estructura puede evolucionar con frecuencia [3].

3.2.3. Plataforma de *streaming* de eventos: *RabbitMQ vs Kafka*

***Kafka* (seleccionada).** *Kafka* se basa en un modelo *Dumb Broker / Smart Consumer*, actuando como un registro distribuido y persistente de eventos. Esta arquitectura permite desacoplar los microservicios, soportar múltiples consumidores independientes y habilitar el reprocesamiento de eventos, lo que resulta especialmente adecuado para arquitecturas orientadas a eventos y patrones como CQRS.

RabbitMQ. RabbitMQ sigue el enfoque *Smart Broker / Dumb Consumer*, delegando en el broker la lógica de enrutamiento y eliminando los mensajes tras su consumo. Este modelo es apropiado para colas de trabajo tradicionales y procesamiento inmediato de tareas, pero limita el multiconsumo y la capacidad de reprocesamiento, aspectos clave en sistemas basados en eventos.

Conclusión. Se elige *Kafka* como plataforma de mensajería por su persistencia de eventos, su capacidad de multiconsumo y su mejor encaje con una arquitectura orientada a eventos, donde los servicios requieren acceso al histórico de cambios de forma desacoplada. [4].

4. Diseño de la arquitectura

En los siguientes apartados, ahondaremos en nuestra propuesta de arquitectura de alto nivel para el MVP de SolidarianID. Presentaremos un diagrama representativo de dicha arquitectura en el que se reflejan las distintas bases de datos propuestas en el apartado anterior, explicaremos los distintos componentes y, por último, comentaremos cómo se integran entre si.

4.1. Diagrama de la arquitectura final en el MVP

En este apartado introduciremos nuestra propuesta de arquitectura de alto nivel para el caso de estudio SolidarianID (en esta primera release[En el MVP]). En la Figura 2 podemos ver dicho diagrama.

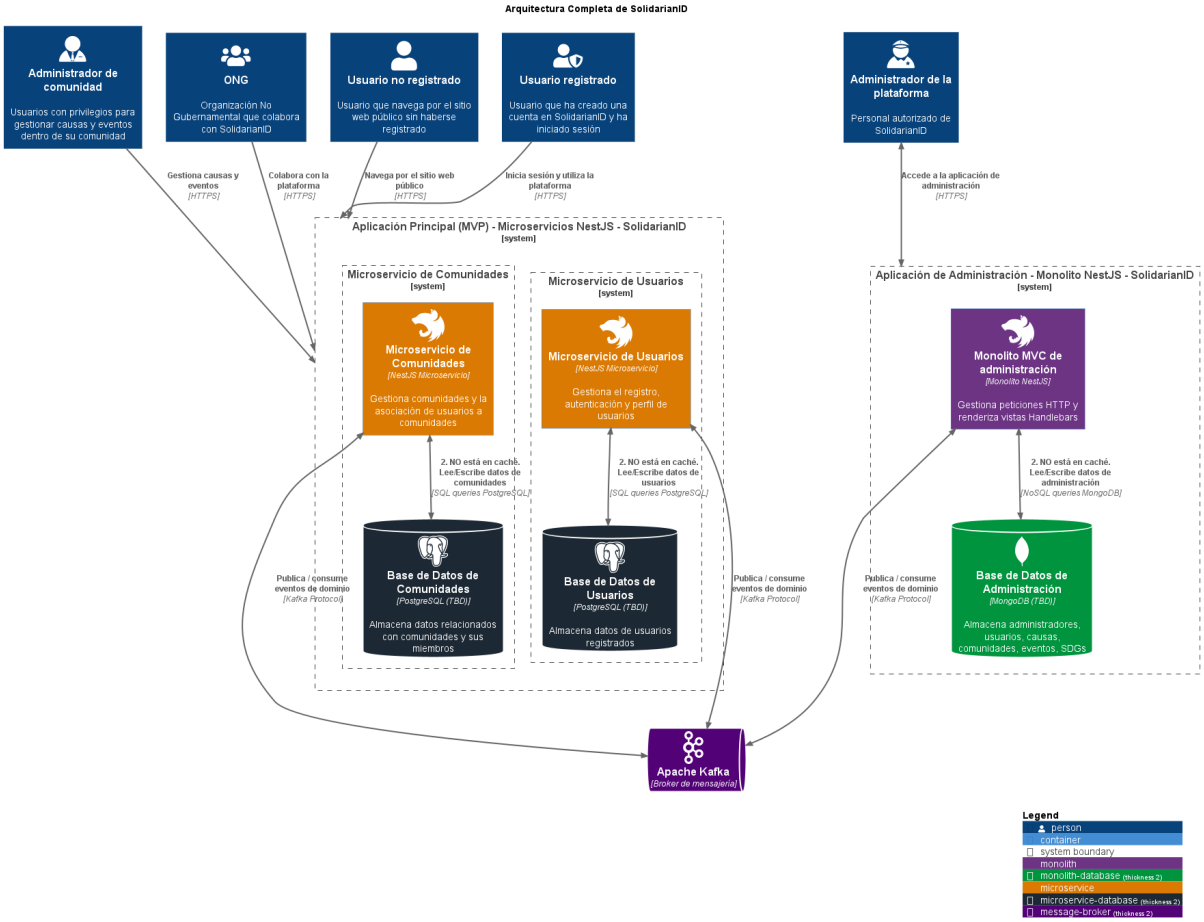


Figura 2: Diagrama de la arquitectura del MVP de SolidarianID

4.2. Explicación de los componentes y su integración

La arquitectura de datos de SolidarianID se ha diseñado separando claramente el almacenamiento transaccional del almacenamiento orientado a lectura y analítica, apoyándose además en una capa de cache para rendimiento y en un bus de eventos para sincronización. El núcleo de la información del sistema se mantiene en PostgreSQL, donde residen las entidades del *core* y sus relaciones, garantizando integridad referencial y consistencia en operaciones de escritura. Este *core* se implementa con un enfoque de *database per service*, de modo que cada microservicio NestJS dispone de su propia base de datos: el microservicio de Usuarios persiste datos de identidad, autenticación y perfil en su PostgreSQL, y el microservicio de Comunidades persiste todo lo relativo a comunidades en PostgreSQL. Esta separación reduce acoplamiento entre dominios y permite evolucionar esquemas de forma independiente sin comprometer la coherencia del resto del sistema.

La integración entre el *core* y las vistas de lectura se realiza mediante Apache Kafka, que se utiliza como registro persistente de eventos de dominio. Cuando ocurren cambios relevantes en los datos transaccionales (creación/actualización de entidades, cambios de estado, etc.), los microservicios publican eventos en Kafka. Estos eventos pueden ser consumidos por otros componentes sin necesidad de acoplamientos directos, permitiendo mantener proyecciones y modelos de lectura actualizados de forma asíncrona. Al conservar el histórico de eventos, Kafka posibilita además el reprocesamiento para reconstruir vistas si cambian los requisitos del dashboard o si se requiere regenerar datos derivados.

La aplicación de administración se apoya en un modelo de datos distinto al transaccional, almacenando en MongoDB proyecciones denormalizadas y agregadas del sistema. En lugar de consultar PostgreSQL para analítica, el monolito de administración consume eventos desde Kafka y materializa en MongoDB documentos optimizados para lectura, como estadísticas, informes, historiales y resúmenes por identificador. Este diseño permite que la estructura de estas vistas evolucione con facilidad sin forzar migraciones complejas en el *core*, aceptando una consistencia eventual adecuada para un contexto de reporting.

En conjunto, el diseño de datos queda organizado en tres capas: una capa de *verdad transaccional* en PostgreSQL por microservicio, y una capa de *lectura analítica* en MongoDB alimentada por eventos Kafka. Esta combinación permite mantener consistencia fuerte en el *core*, flexibilidad en las vistas administrativas y un flujo de integración desacoplado y escalable, aunque el modelo vaya evolucionando con el tiempo.

5. Implementación de la arquitectura

SolidarianID se ha implementado siguiendo una arquitectura de microservicios con persistencia políglota, donde cada servicio gestiona su propio almacenamiento y la comunicación entre dominios se apoya en eventos. En el MVP nos hemos centrado en el *administrador* (panel y flujo de validación/denegación de solicitudes), priorizando: (i) un almacenamiento flexible para solicitudes, (ii) una base de datos transaccional para datos operacionales y (iii) un bus de eventos para desacoplar servicios y evitar dependencias síncronas en operaciones administrativas [4, 5].

5.1. Descripción de la implementación de cada sistema de base de datos

Los microservicios *Users* y *Communities* persisten en PostgreSQL por su idoneidad para datos estructurados con integridad referencial y necesidades transaccionales (garantías ACID y concurrencia con MVCC [Multi-Version Concurrency Control] y niveles de aislamiento), facilitando operaciones seguras con alta concurrencia [2, 6, 7]. En el MVP la persistencia se apoya en un ORM (TypeOrm); en un despliegue real se reforzaría con migraciones versionadas e índices planificados para carga elevada.

El microservicio *Admin* utiliza MongoDB para el ciclo de vida de *solicitudes*, al ser un modelo documental apropiado para información heterogénea y estructuras anidadas, reduciendo la necesidad de `JOIN` y facilitando la evolución del esquema [3, 8]. Esto simplifica lecturas del panel (estado, motivo y datos asociados) y permite endurecer garantías mediante configuraciones de lectura/escritura, replicación y transacciones cuando sea necesario [8].

Para la integración entre servicios se emplea Apache Kafka como plataforma de streaming de eventos, propagando cambios relevantes (usuarios, comunidades, solicitudes) de manera asíncrona. Este enfoque mejora el desacoplamiento y permite escalar consumidores y procesar en paralelo mediante particiones, algo especialmente útil ante picos de carga [4, 5].

5.2. Mecanismos implementados o que se podrían implementar

El MVP académico valida el enfoque (microservicios + persistencia políglota + eventos), pero una aplicación con alto flujo de usuarios requiere complementar la implementación con mecanismos explícitos de consistencia, replicación/particionamiento y tolerancia a fallos. A continuación se resumen mejoras típicas (algunas no implementadas por alcance).

5.2.1. Consistencia de datos

En una arquitectura con persistencia políglota, la consistencia debe analizarse en tres niveles: dentro de cada base de datos (intra-servicio), dentro del modelo documental (unidad de agregación) y entre microservicios (propagación por eventos). En el caso relacional, PostgreSQL ofrece consistencia fuerte gracias a transacciones ACID y control de concurrencia mediante MVCC, de modo que múltiples lecturas/escrituras concurrentes pueden ejecutarse sin bloquear de forma agresiva y manteniendo invariantes del dominio [2, 6]. Además, los niveles de aislamiento permiten ajustar el equilibrio entre rendimiento y anomalías tolerables (por ejemplo, *Read Committed* o *Serializable*), lo que resulta clave cuando el sistema crece y aparecen conflictos de concurrencia [7]. En un sistema de alto flujo (más allá del MVP), esta consistencia se refuerza típicamente con: (i) restricciones (*PK/FK*, *unique*), (ii) validaciones a nivel de aplicación y (iii) reintentos controlados cuando se producen errores de serialización en transacciones contendidas [7].

Para el microservicio *Admin*, el uso de MongoDB encaja con lo visto en el tema de bases documentales: la **unidad natural de consistencia** es el documento, ya que las actualizaciones sobre un mismo documento son atómicas, lo que simplifica mantener coherencia en agregados como *Solicitud* (estado, motivo, metadatos y datos anidados para revisión) [8]. En despliegues con *replica set*, y cuando el caso de uso lo exige, pueden emplearse transacciones para coordinar cambios multi-documento o multi-colección, aunque el diseño recomendado en documental suele intentar minimizar esa necesidad mediante un modelado adecuado (embebido vs referencia) [8]. Este enfoque se alinea con la motivación de elegir MongoDB frente a alternativas: flexibilidad de esquema y eficiencia en lecturas de estructuras complejas, aceptando que la consistencia puede configurarse según necesidades de disponibilidad/rendimiento [3, 8].

Entre microservicios, la consistencia pasa a ser predominantemente **eventual**: Kafka desacopla publicadores y consumidores y los cambios se propagan como eventos que los servicios procesan de forma asíncrona, convergiendo con el tiempo [4, 5]. Para un entorno real con mucho volumen, donde pueden existir reintentos y reprocesamientos, se suelen incorporar mecanismos no implementados en el MVP:

- **Procesamiento idempotente** en consumidores (deduplicación por *eventId* u offset aplicado) para tolerar entregas repetidas sin corromper estado [5].
- **Coordinación distribuida** mediante coreografía/orquestación (por ejemplo, uso del patrón *SAGA* con distribuido) para operaciones que afectan a varios servicios sin recurrir a transacciones distribuidas bloqueantes[5].
- **Publicación fiable** (por ejemplo, *outbox* junto a PostgreSQL) para reducir el riesgo de discrepancia entre "dato persistido" y "evento emitido", apoyándose en propiedades transaccionales del relacional [7, 5].

5.2.2. Replicación y particionamiento

Cuando el sistema debe soportar alta disponibilidad y crecimiento, replicación y particionamiento se convierten en mecanismos estructurales. En PostgreSQL, la documentación oficial describe enfoques de alta disponibilidad basados en replicación y *hot standby*, donde réplicas pueden mantenerse listas para atender lecturas o promoverse ante un fallo del primario [9, 10]. En entornos gestionados, es habitual desplegar *read replicas* para escalar la carga de lectura sin sobrecargar el nodo principal, algo especialmente útil para paneles administrativos con consultas repetitivas y filtrados frecuentes [11]. Además, PostgreSQL soporta **particionamiento declarativo** para tablas grandes, reduciendo el volumen de datos explorado por consulta mediante *partition pruning* (por ejemplo, históricos por rango temporal) [12]. Por último, la **replicación lógica** permite replicar cambios a nivel lógico (tablas/publicaciones) y resulta útil cuando se desea distribuir subconjuntos de datos o habilitar integraciones que consumen cambios sin replicar toda la instancia [13].

En MongoDB, la replicación se articula mediante *replica sets* con elección automática de primario ante fallo (*failover*), aumentando la disponibilidad del servicio de administración [8]. Para escalar horizontalmente, el *sharding* distribuye documentos según una *shard key* en fragmentos, permitiendo repartir carga y almacenamiento entre nodos; en el caso del

Admin, colecciones de solicitudes podrían particionarse por claves de acceso frecuente (por ejemplo, `status` o `userId`) para minimizar consultas dispersas [8]. Este tipo de particionamiento es coherente con el objetivo de soportar grandes volúmenes en bases no relacionales y con la comparación general de enfoques de escalado en sistemas distribuidos [3].

En Kafka, el mecanismo central de escalado es la **partición** de los tópicos: las particiones habilitan paralelismo de consumo (grupos de consumidores) y, junto a la **replicación** de particiones, aportan durabilidad y tolerancia a fallos de brokers [5]. En un despliegue real se dimensiona el número de particiones según el paralelismo deseado y se configura el factor de replicación para garantizar disponibilidad; esta orientación a *event streaming* es una de las diferencias clave frente a colas tradicionales, tal y como se discute en comparativas entre Kafka y sistemas orientados a mensajería [4, 5].

5.2.3. Escalabilidad y tolerancia a fallos

La escalabilidad en un sistema con alto flujo de usuarios se aborda en tres capas: cómputo (réplicas de servicios), datos (capacidad de lectura/escritura en BDs) y mensajería (capacidad de ingesta/procesamiento de eventos). En PostgreSQL, además del escalado vertical, se combinan estrategias de alta disponibilidad con réplicas y *standbys* para conmutación ante fallos [9]. Para escalar lecturas se recurre a *read replicas*, especialmente en despliegues gestionados, y se refuerza la operación con prácticas de durabilidad como backups automatizados y configuraciones administradas orientadas a resiliencia [11, 14]. Un punto crítico cuando crece el número de usuarios concurrentes es el exceso de conexiones: introducir **pooling** con PgBouncer reduce el coste de gestionar miles de conexiones y evita saturar el servidor de base de datos [15]. Estas decisiones se alinean con el uso de PostgreSQL como motor transaccional robusto para cargas OLTP (Online Transaction Processing, tipo de carga típica de una aplicación "viva" (web/móvil) donde el sistema hace muchas operaciones pequeñas y rápidas sobre la base de datos) exigentes [2].

En MongoDB, la tolerancia a fallos se apoya en *replica sets* (reelección de primario) y la escalabilidad horizontal en *sharding*, lo que permite repartir tanto almacenamiento como throughput cuando crece el volumen de documentos o el número de operaciones por segundo [8]. En un panel de administración con mucha actividad (altas/validaciones/consultas), esta combinación facilita absorber picos de lectura y escritura sin degradar drásticamente la disponibilidad, manteniendo además la flexibilidad del modelo documental [3, 8].

En Kafka, el escalado se consigue incrementando particiones y consumidores dentro de un grupo, lo que permite paralelizar el procesamiento; la tolerancia a fallos se basa en replicación y reequilibrio de consumidores cuando instancias fallan o se despliegan nuevas réplicas [5]. En un sistema real, y aunque no se haya implementado en el MVP, también se suelen añadir mecanismos operativos coherentes con el procesamiento escalable: *dead-letter topics* para mensajes problemáticos, políticas de reintentos con *backoff* y observabilidad del *lag* para detectar cuellos de botella antes de que impacten a la experiencia del administrador [5]. Además, la elección de Kafka frente a otras alternativas de mensajería se justifica por su orientación a *streaming* y su capacidad para sostener flujos

continuos de eventos a gran escala [4]. En nuestro caso, en lo relativo a Kafka únicamente hemos implementado una partición por tópico, aunque, cabe destacar que nuestra implementación genérica de kafka también sirve para garantizar la consumición ordenada de eventos cuando el número de particiones escala horizontalmente de manera proporcional al número de consumidores de un mismo grupo en un tópico (ya que como bien sabemos, kafka tiene un limite de un consumidor / partición). Esto se debe a nuestra implementación genérica de un publicador Kafka el cual utiliza como "key" de los mensajes el `UUID` de la entidad asociada que se notifica por evento que ha sido modificada.

6. Consultas y procesamiento de datos

6.1. Ejemplos de consultas para los casos de uso principales de MongoDB

En este apartado se detallan las consultas más relevantes implementadas en la arquitectura, mostrando su propósito y la traducción final realizada por el ODM (Mongoose) hacia la base de datos.

6.1.1. Listado de comunidades pendientes (findPendingCommunityRequests)

Listar todas las comunidades pendientes de aprobación de forma paginada, incluyendo todas las causas asociadas a cada comunidad. Esto permite a los administradores visualizar la solicitud completa (comunidad y sus causas iniciales) antes de aprobarlas o rechazarlas. Véase el Código 1.

```
requests.aggregate([
  { $match: { status: "pending", type: "community" } },
  {
    $lookup: {
      from: "requests",
      let: { communityId: "$_id" },
      pipeline: [
        {
          $match: {
            $expr: { $eq: ["$communityId", "$$communityId"] },
            type: "cause",
          },
        },
      ],
      as: "causes",
    },
  },
  { $skip: 0 },
  { $limit: 10 },
]);
```

Código 1: *Query* de agregación para comunidades pendientes

6.1.2. Agrupación de Causas por ODS (findCausesGroupByOds)

Mostrar en el dashboard un gráfico de distribución que indique cuántas causas aprobadas existen para cada Objetivo de Desarrollo Sostenible (ODS). Es un KPI crítico para medir el impacto social de la plataforma. Véase el Código 2.

```
requests.aggregate([
  { $match: { $and: [{ type: "cause" }, { status: "approved" }] } },
  { $unwind: "$ods" },
  { $group: { _id: "$ods", causeCount: { $sum: 1 } } },
  { $project: { _id: 0, ods: "$_id", causeCount: 1 } },
  { $sort: { ods: 1 } },
]);
```

Código 2: *Query* para agrupar causas por ODS

6.1.3. Agrupación de Usuarios por País (findUsersGroupByCountry)

Visualizar en el dashboard los 6 países con mayor número de voluntarios registrados, permitiendo identificar la penetración geográfica de la plataforma. Véase el Código 3.

```
users.aggregate([
  { $group: { _id: "$location.countryCode", userCount: { $sum: 1 } } },
  { $sort: { userCount: -1 } },
  { $project: { _id: 0, country: "$_id", userCount: 1 } },
  { $limit: 6 },
]);
```

Código 3: *Query* para agrupar usuarios por país

6.1.4. Listado de causas pendientes (findPaginated / findPendingCauseRequests)

Listar las causas pendientes de validación, asegurando que solo se muestren aquellas pertenecientes a comunidades que ya han sido aprobadas (regla de negocio de integridad). Véase el Código 4.

```
requests.aggregate([
  { $match: { status: "approved", type: "community" } },
  {
    $lookup: {
      from: "requests",
      let: { communityId: "$_id" },
      pipeline: [
        {
          $match: {
            $expr: { $eq: ["$communityId", "$$communityId"] },
            status: "pending",
            type: "cause",
          },
        },
      ],
      as: "causes",
    },
  },
  { $unwind: "$causes" },
  { $group: { _id: null, count: { $sum: 1 } } },
]);
```

Código 4: *Query* de filtrado y conteo para causas pendientes

6.2. Ejemplos de consultas para los casos de uso principales de PostgreSQL

6.2.1. Ranking de comunidades por usuarios (findOrderByMostUsers)

Mostrar un ranking de las comunidades más populares ordenadas por número de miembros. Se usa en dashboards para identificar las comunidades con mayor participación. Véase el Código 5.

```
SELECT
c.id AS "communityId",
c.name AS "communityName",
COUNT(cm."userId")::int AS "userCount"
FROM community c
INNER JOIN community_user cm ON cm."communityId" = c.id
GROUP BY c.id
ORDER BY COUNT(cm."userId")::int DESC
LIMIT :ranking_limit;
```

Código 5: *Query* de ranking de comunidades por usuarios

6.2.2. Causas por ID de comunidad (findAllByCommunityId)

Obtener todas las causas solidarias asociadas a una comunidad específica. Se utiliza cuando un usuario navega a una comunidad y quiere ver solo las causas de esa comunidad, filtrando por la clave foránea correspondiente. Véase el Código 6.

```
SELECT "CauseEntity"."id" AS "CauseEntity_id",
"CauseEntity"."name" AS "CauseEntity_name",
"CauseEntity"."description" AS "CauseEntity_description",
"CauseEntity"."communityId" AS "CauseEntity_communityId",
"CauseEntity"."status" AS "CauseEntity_status"
FROM "causes" "CauseEntity"
WHERE "CauseEntity"."communityId" = $1;
```

Código 6: *Query* de listado de causas por comunidad

6.2.3. Verificar existencia por móvil (existsByMobilePhone)

Validar si un número de teléfono móvil ya está registrado en el sistema antes de permitir que un nuevo usuario se registre. Previene duplicados garantizando la unicidad del campo `MobilePhone` mediante un conteo eficiente. Véase el Código 7.

```
SELECT COUNT(1) AS "cnt"
FROM "tbl_Usuarios" "UserEntity"
WHERE "UserEntity"."MobilePhone" = $1;
```

Código 7: *Query* de verificación de existencia por móvil

6.2.4. Estadística de nuevos miembros (findJoinedUsersLastMonths)

Calcular el número de nuevos usuarios que se han unido a las comunidades en los últimos meses, agrupados por mes. Esta métrica permite visualizar la tendencia de crecimiento de la comunidad en el dashboard de administración. Véase el Código 8.

```
SELECT
DATE_TRUNC('month', cm."joinedAt") AS "month",
COUNT(cm."userId")::int AS "count"
FROM community_user cm
WHERE cm."joinedAt" >= :startDate
GROUP BY DATE_TRUNC('month', cm."joinedAt")
ORDER BY "month" ASC;
```

Código 8: *Query* de usuarios unidos en los últimos meses

6.3. Explicación de cómo cada consulta optimiza el rendimiento

Con la implementación de estas consultas, hemos podido validar en la práctica la eficacia de nuestra estrategia de persistencia políglota. Al desarrollar el sistema, hemos diseñado las consultas de PostgreSQL (Sección 6.2) priorizando la integridad referencial y la consistencia transaccional mediante el uso estricto de relaciones y `JOINs`. Simultáneamente, en el módulo de administración, hemos explotado las capacidades del `aggregate framework` de MongoDB (Sección 6.1) para generar métricas y vistas analíticas de forma ágil, evitando así la complejidad de realizar estos cálculos sobre el modelo relacional normalizado.

Por un lado, desde el punto de vista del rendimiento en la capa relacional, el uso de TypeORM aporta optimizaciones estructurales al generar internamente índices asociados a las claves primarias de todas las tablas, lo que acelera de forma directa las operaciones de consulta y acceso por identificador. Adicionalmente, se han definido índices explícitos en aquellas tablas intermedias que materializan relaciones de muchos a muchos, como `community_user` mostrada en el fragmento de código 9, que gestiona los permisos y roles de los usuarios dentro de cada comunidad. Dado que esta tabla actúa como una *ACL* (control de acceso) y será consultada de forma recurrente para validar autorizaciones, estos índices permiten optimizar significativamente las búsquedas (por ejemplo, por `communityId` y `userId`), reduciendo la latencia en las operaciones críticas de validación de permisos.

```
 @Index(['communityId', 'userId', 'role'])
@Entity('community_user')
export class CommunityUserEntity {
  @PrimaryColumn()
  communityId: string;

  @PrimaryColumn()
  userId: string;

  // Otros campos
}
```

Código 9: Ejemplo de la tabla COMMUNITY_USER

Por otro lado, para las consultas mencionadas para solicitudes, al tratarse de un flujo operativo centrado en la lectura y filtrado (por `tipo`, `estado` y `userId`) (Vease código 10), se han definido índices explícitos sobre dichos campos, lo que permite resolver de forma eficiente listados de moderación y consultas del historial del usuario, mejorando los tiempos de respuesta incluso ante un crecimiento del volumen de solicitudes.

```
RequestSchema.index({ type: 1 });
RequestSchema.index({ status: 1 });
RequestSchema.index({ userId: 1 });
```

Código 10: Ejemplo de indices en RequestSchema

Esta separación de responsabilidades nos ha permitido que el núcleo operacional mantenga su robustez, mientras que el dashboard analítico ofrece tiempos de respuesta inmediatos. Hemos comprobado que delegar la lógica relacional fuerte en PostgreSQL y la proyección de datos estadísticos en MongoDB no solo simplifica el desarrollo de cada microservicio, sino que también nos asegura que la carga analítica no penalice el rendimiento de las operaciones críticas del usuario.

7. Descripción de configuración del Docker e instrucciones de despliegue

El enlace a GitHub proporcionado apunta hacia la rama `develop`, donde hemos incorporado los últimos cambios. En el directorio `/ado` se puede encontrar una guía en el *markdown* `LocalDeployment.md` para lanzar el stack de las aplicaciones y poder probarlo. Además, se incluyen dos scripts para probar tanto las consultas definidas en el punto 6 como algunas más relevantes que no cabían en esta memoria por espacio. También se incluyen sendos respectivos *markdowns* para esos scripts que incluyen todo el detalle de las consultas, como las generan los ORM (Mongoose y TypeORM) a partir de métodos de repositorio y qué buscan en la base de datos.

Dicha información se puede encontrar en la raíz de nuestro repositorio [1] en la ruta `ado/`.

8. Conclusiones

La realización de este trabajo ha supuesto un aprendizaje significativo en nuestra formación como ingenieros informáticos, al permitirnos aplicar de manera integrada y práctica los conceptos teóricos de arquitectura de datos estudiados durante la asignatura. El diseño e implementación de la arquitectura de datos de *SolidarianID* nos ha obligado a analizar el dominio con rigor, identificar correctamente las entidades y relaciones fundamentales y tomar decisiones técnicas fundamentadas en requisitos reales de consistencia, escalabilidad y rendimiento.

Uno de los principales aportes formativos ha sido la experiencia en la toma de decisiones arquitectónicas justificadas, evaluando distintas alternativas tecnológicas y comprendiendo sus implicaciones a medio y largo plazo. La comparación entre bases de datos relacionales y documentales, así como entre diferentes plataformas de mensajería, nos ha permitido interiorizar que no existe una solución única válida para todos los problemas, sino que el valor de una arquitectura reside en su adecuación al contexto y a los casos de uso concretos.

Asimismo, el trabajo ha reforzado nuestra comprensión de arquitecturas modernas basadas en microservicios, persistencia políglota y comunicación orientada a eventos. La integración de *PostgreSQL* como núcleo transaccional, *MongoDB* como base de datos orientada a lectura y analítica, y *Kafka* como mecanismo de desacoplamiento entre servicios, nos ha proporcionado una visión práctica de cómo se diseñan sistemas distribuidos robustos, tolerantes a fallos y preparados para evolucionar. Este enfoque nos ha permitido asimilar conceptos clave como consistencia eventual, proyecciones de lectura, desacoplamiento entre dominios y separación de responsabilidades a nivel de datos.

En conjunto, este trabajo no solo ha servido para consolidar conocimientos técnicos en arquitectura de datos, sino también para fortalecer una mentalidad de diseño orientada a la calidad, la escalabilidad y la mantenibilidad de los datos en el desarrollo de software.

Referencias

[1] Equipo 3. *Repositorio de GitHub del equipo 3*. 2026. URL: https://github.com/fsoladur/SolidarianID.

[2] Amazon Web Services. *¿Cuál es la diferencia entre MySQL y PostgreSQL?* https://aws.amazon.com/es/compare/the-difference-between-mysql-vs-postgresql/. Accedido: 13 de diciembre de 2025. 2025.

[3] Amazon Web Services. *¿Cuál es la diferencia entre Apache Cassandra y MongoDB?* https://aws.amazon.com/es/compare/the-difference-between-cassandra-and-mongodb/. Accedido: 13 de diciembre de 2025. 2025.

[4] Amazon Web Services. *¿Cuál es la diferencia entre RabbitMQ y Apache Kafka?* https://aws.amazon.com/es/compare/the-difference-between-rabbitmq-and-kafka/. Accedido: 13 de diciembre de 2025. 2025.

[5] Catalina Martínez Costa. *Tema 8. Procesamiento escalable de datos*. Material docente (MISum, Arquitectura de Datos, Universidad de Murcia). Accedido: 7 de enero de 2026. 2025.

[6] PostgreSQL Global Development Group. *Introduction to Concurrency Control (MVCC)*. https://www.postgresql.org/docs/current/mvcc-intro.html. Accedido: 7 de enero de 2026. 2025.

[7] PostgreSQL Global Development Group. *Transaction Isolation*. https://www.postgresql.org/docs/current/transaction-iso.html. Accedido: 7 de enero de 2026. 2025.

[8] Catalina Martínez Costa. *Tema 3. Bases de datos de documentos*. Material docente (MISum, Arquitectura de Datos, Universidad de Murcia). Accedido: 7 de enero de 2026. 2025.

[9] PostgreSQL Global Development Group. *High Availability, Load Balancing, and Replication*. https://www.postgresql.org/docs/current/high-availability.html. Accedido: 7 de enero de 2026. 2025.

[10] PostgreSQL Global Development Group. *Hot Standby*. https://www.postgresql.org/docs/current/hot-standby.html. Accedido: 7 de enero de 2026. 2025.

[11] Amazon Web Services. *Working with read replicas for Amazon RDS for PostgreSQL*. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_PostgreSQL.Replication.ReadReplicas.html. Accedido: 7 de enero de 2026. 2025.

[12] PostgreSQL Global Development Group. *Table Partitioning*. https://www.postgresql.org/docs/current/ddl-partitioning.html. Accedido: 7 de enero de 2026. 2025.

[13] PostgreSQL Global Development Group. *Logical Replication*. https://www.postgresql.org/docs/current/logical-replication.html. Accedido: 7 de enero de 2026. 2025.

[14] Amazon Web Services. *Amazon RDS for PostgreSQL*. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_PostgreSQL.html. Accedido: 7 de enero de 2026. 2025.

[15] PgBouncer Authors. *PgBouncer command-line usage*. https://www.pgbouncer.org/usage.html. Accedido: 7 de enero de 2026. 2025.

A. Arquitectura completa de SolidarianID del grupo 3

En este anexo se presenta la *arquitectura completa objetivo* de *SolidarianID* (más allá del MVP), poniendo el foco en la *arquitectura de datos* y en cómo se distribuye, sincroniza y optimiza la información en una solución basada en microservicios y persistencia políglota. La Figura 3 sintetiza los contenedores principales, sus almacenes de datos y los flujos de comunicación (HTTP y eventos) entre dominios.

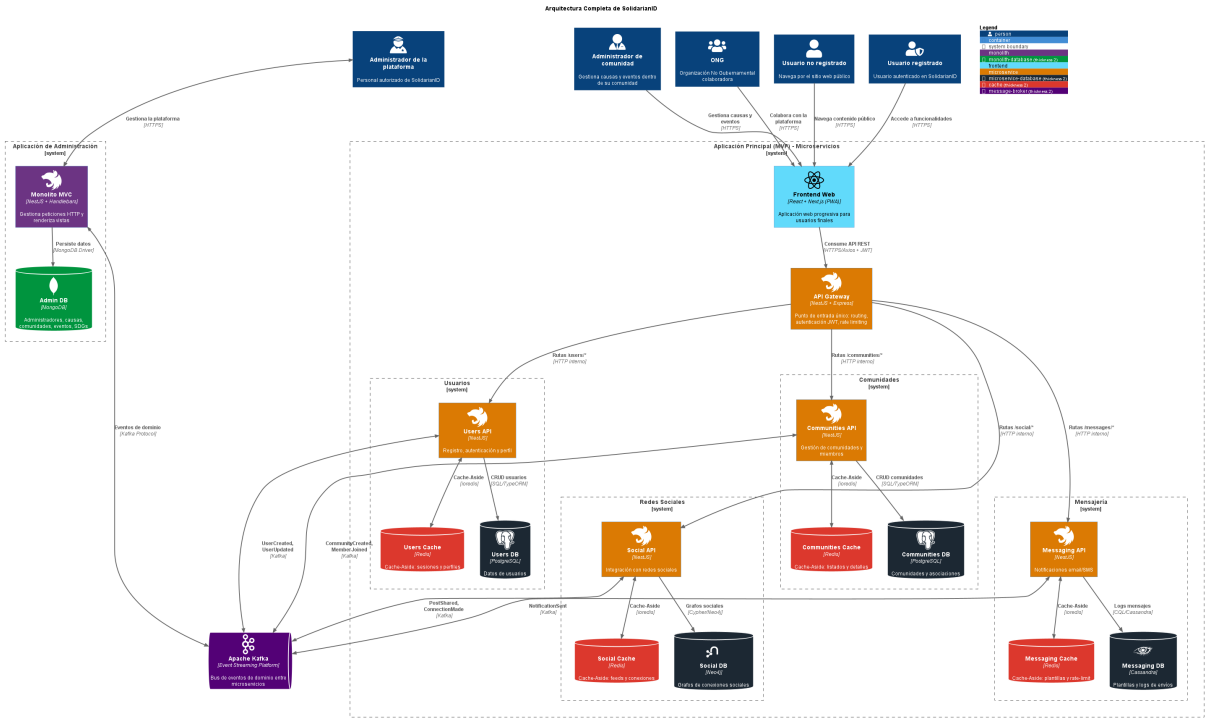


Figura 3: Arquitectura final y completa planteada para SolidarianID: microservicios, bases de datos, cachés y bus de eventos.

A.1. Principios de arquitectura de datos

La solución sigue tres principios que estructuran todas las decisiones de datos:

- *Database per service*: cada microservicio es propietario de su almacenamiento y de su modelo de datos, evitando acoplamiento a nivel de esquema y permitiendo evolucionar cada contexto de manera independiente.
- *Persistencia políglota*: se selecciona el motor de persistencia según el patrón de acceso dominante (OLTP relacional, proyecciones documentales, grafos, logs de alta escritura, etc.), priorizando consistencia y rendimiento por caso de uso.
- *Integración asíncrona mediante eventos*: Apache Kafka actúa como registro persistente de eventos de dominio, facilitando sincronización eventual entre servicios, desacoplamiento y la posibilidad de reprocesamiento para reconstrucción de vistas derivadas.

A.2. Capa transaccional del *core*: Users y Communities

El *core* funcional se apoya en dos microservicios transaccionales:

- *Users API*: gestiona registro, autenticación y perfil. Persiste en *Users DB (PostgreSQL)* como fuente de verdad transaccional, aprovechando integridad referencial y garantías ACID.
- *Communities API*: gestiona comunidades, membresías y asociaciones. Persiste en *Communities DB (PostgreSQL)* para mantener coherencia en relaciones y operaciones de escritura concurrentes.

En ambos casos se incorpora una capa de *caché Redis* con patrón *cache-aside*:

- *Users Cache (Redis)*: acelera lecturas frecuentes (por ejemplo, sesiones/perfiles).
- *Communities Cache (Redis)*: optimiza listados y detalles de comunidades.

Este enfoque reduce presión sobre las bases transaccionales y mejora latencias en operaciones dominadas por lectura, manteniendo la consistencia fuerte donde es necesaria (escrituras en PostgreSQL) y aceptando expiración/invalidez controlada en caché.

A.3. Aplicación de administración y proyecciones analíticas

La *aplicación de administración* se implementa como un monolito MVC (NestJS + Handlebars) orientado a moderación, reporting y métricas. Desde el punto de vista de datos, su rol no es el de *source of truth* del dominio transaccional, sino el de *materializar vistas de lectura* en una base documental:

- *Admin DB (MongoDB)*: almacena documentos denormalizados orientados al panel (administradores y proyecciones de contenido como comunidades/causas/eventos/ODS), facilitando agregaciones y evolución del esquema sin migraciones complejas del core.
- *Kafka* alimenta estas proyecciones: el monolito consume eventos de dominio (p. ej., creación/actualización de usuarios, creación de comunidades, cambios relevantes) y actualiza los documentos del dashboard de forma asíncrona.

De este modo, se mantiene *consistencia fuerte* en el núcleo transaccional y *consistencia eventual* en la capa analítica/administrativa, adecuada para escenarios de reporting.

A.4. Dominios extendidos: redes sociales y mensajería

La arquitectura completa incorpora dominios adicionales que justifican la persistencia políglota:

Redes sociales (Social API). Este dominio maneja relaciones y conexiones entre usuarios, que encajan naturalmente en un modelo de grafos:

- *Social DB (Neo4j)*: almacena el grafo social (conexiones, relaciones, recomendaciones).
- *Social Cache (Redis)*: cache-aside para *feeds* y conexiones, reduciendo coste de lecturas repetidas.
- *Eventos Kafka* (p. ej., `PostShared`, `ConnectionMade`) permiten propagar cambios y mantener vistas/contadores derivados sin acoplamientos síncronos.

Mensajería y notificaciones (Messaging API). Este dominio prioriza trazabilidad y alto volumen de escritura de logs de envíos:

- *Messaging DB (Cassandra)*: orientada a escrituras rápidas y escalado horizontal para *logs* de notificaciones/envíos.
- *Messaging Cache (Redis)*: cache-aside para plantillas y mecanismos de *rate-limit*, reduciendo latencia y protegiendo al sistema de picos.
- *Eventos Kafka* (p. ej., `NotificationSent`) permiten auditoría y sincronización con otros servicios interesados (analítica, administración, etc.).

A.5. API Gateway y gobierno del acceso a datos

La arquitectura incorpora un *API Gateway* (NestJS + Express) como punto de entrada para el *Frontend Web (React + NextJS PWA)*. Aunque el *gateway* no persiste datos, sí gobierna el acceso y exposición de los mismos:

- *Routing* hacia `/users`, `/communities`, `/social` y `/messages`.
- *Autenticación JWT* y políticas de acceso.
- *Rate limiting*, reduciendo riesgo de abuso y protegiendo los almacenes aguas abajo (PostgreSQL/Redis/Cassandra/Neo4j).

A.6. Consistencia e integración global

En conjunto, la arquitectura completa establece un equilibrio explícito:

- *Consistencia fuerte intra-servicio* en bases transaccionales (PostgreSQL) y en unidades atómicas documentales (MongoDB).
- *Consistencia eventual inter-servicio* mediante Kafka, habilitando desacoplamiento y escalabilidad.

- *Rendimiento* mediante cachés cache-aside (Redis) en dominios con fuerte componente de lectura o con requisitos de baja latencia.

Este diseño extiende de manera natural la arquitectura del MVP descrita en el cuerpo del documento, manteniendo sus fundamentos (microservicios, persistencia políglota y bus de eventos) y ampliándolos para cubrir dominios con patrones de datos específicos (grafos sociales y mensajería a gran escala).